

Proyecto corto III: División de enteros

Dr.-Ing Alfonso Chacón-Rodríguez
Dr.-Ing Oscar Caravaca-Mora
Abner López-Méndez
Felipe Zumbado-Rodríguez

1. Introducción

El diseño de sistemas digitales requiere habilidad de implementación de algoritmos por medio de circuitos lógicos. Muchos algoritmos en la práctica usan iteraciones, segmentación (pipelining) o bucles que a la hora de traducirlos a implementaciones de lógica booleana, surge la necesidad de un control lógico que habilite el correcto flujo de datos en circuito. Asimismo, las interfaces de bloque a bloque se diseñan con protocolos de bus para ayudar a estandarizar la comunicación entre unidades. Estos protocolos de bus facilitan las pruebas unitarias sobre bloques porque toda unidad se puede controlar de una manera similar.

En este proyecto busca introducir la implementación de algoritmos al estudiante, por medio del diseño de una unidad de cálculo de división de enteros. Y de la misma forma, esta unidad deberá respetar un protocolo de bus para su correcto funcionamiento.

2. Objetivo general

Introducir al estudiante a la implementación de algoritmos por medio de máquinas de estados complejas

3. Objetivos específicos

1. Elaborar una implementación de un diseño digital en una FPGA.
2. Construir un *testbench* básico para validar las especificaciones del diseño.
3. Implementar un algoritmo de división de enteros con una Máquina de estados con técnicas avanzadas.
4. Coordinación de trabajo en equipo mediante el uso de herramientas de control de versiones.
5. Practicar planificación de tareas para trabajo de grupo.

4. Especificación del diseño

Se solicita el desarrollo de una **unidad de división de enteros** sin signo, con un divisor decimal representable en un máximo de 4 bits y un dividendo decimal representable en un máximo de 6 bits, basándose en el algoritmo visto en clase según [3], sección 5.2.7. Esto limitará la división entera a un máximo de 63_D entre 15_D . El sistema deberá entregar como resultado el cociente y residuo de la operación.

El sistema deberá construirse según las pautas fundamentales de diseño digital sincrónico. El circuito constará de tres bloques de constructivos o subsistemas:

1. Subsistema de lectura de datos.
2. Subsistema de cálculo de división de enteros.

3. Subsistema de conversión de binario a representación BCD.
4. Subsistema de despliegue en display de 7 segmentos.

4.1. Subsistema de lectura

El subsistema de lectura tomará de un teclado hexadecimal los datos de entrada de dos números en formato decimal: un divisor y un dividendo.

Utilice el subsistema ya implementado en el proyecto corto II. Se sugiere incluir una tecla de borrado en el caso de que no se haya implementado la misma en el proyecto corto anterior.

4.2. Subsistema de cálculo de la división entera

1. Este sistema recibe el dividendo A y el divisor B del subsistema de lectura que han sido ya convertidos a su formato binario de 6 y 4 bits respectivamente.
2. La operación de división se iniciará cuando el subsistema de lectura le indique a este subsistema que dividendo y divisor son válidos por medio de una bandera *valid*.
3. Este bloque indicará al siguiente bloque consecutivo cuando el resultado de la división sea estable para desplegar el cociente y residuo de la operación por medio de una bandera **done**.

4.3. Subsistemas de conversión de binario a representación BCD y viceversa

Utilice los sistemas ya implementados en el proyecto corto II para esta etapa.

4.4. Subsistema de despliegue en display de 7 segmentos.

Utilice el subsistema desarrollado en el proyecto anterior. Recuerde eso sí utilizar transistores para el manejo de los 7 segmentos, y no directamente desde los pines de la FPGA.

Añada además una provisión ya sea en el teclado o mediante un botón aparte, que permita seleccionar entre el despliegue del cociente y el residuo.

5. Algoritmo de división de enteros

El algoritmo de división de enteros se explica con detalle en la sección 5.7.2 de [3] y será visto en clase. Note que ésta es una implementación directa combinacional. El algoritmo general se describe acá de nuevo en pseudocódigo en la Fig. 1.

```

R' = 0
for i = N-1 to 0
    R = {R' << 1, Ai}
    D = R - B
    if D < 0 then    Qi = 0, R' = R    // R < B
    else            Qi = 1, R' = D    // R ≥ B
R = R'
```

Figura 1: Algoritmo iterativo de división de enteros, según [3].

Se provee un diagrama de bloques de un arreglo que resuelve el algoritmo anterior en Fig. 2 para el caso de una división con divisor y dividendo de 4 bits. Este bloque es totalmente combinacional. Nótese que el retardo de este divisor de N bits aumenta proporcionalmente a N^2 porque el acarreo debe propagarse

por todas las N etapas consecutivas antes de que se determine el signo y el multiplexor seleccione R o D . Esto se repite para todas las N filas. Una manera de acelerar esto es usar una estructura de segmentación o pipelining entre cada fila. Ello significa que se requerirán una latencia de cuatro ciclos de reloj para resolver la operación, a cambio de cortar el camino crítico y acelerar el reloj a la frecuencia esperada de 27 MHz.

En la Fig. 3 se detalla lo que existe en cada bloque para implementar la resta y decidir si se acepta la operación o se mantiene el residuo anterior.

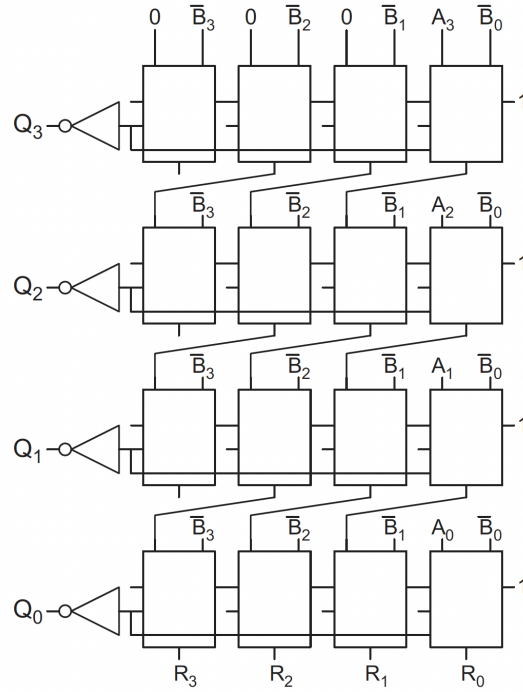


Figura 2: Diagrama de bloques del circuito en forma de arreglo que ejecuta la división de dos números de 4 bits en un solo bloque combinacional, según se describe en 1. Noten que la ruta crítica puede ser de muchos sumadores.

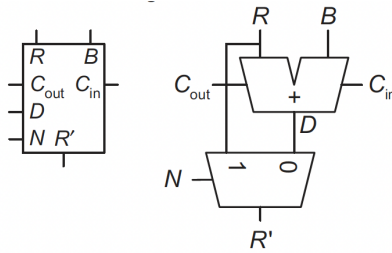


Figura 3: Bloque fundamental para realizar la resta y selección de residuo.

6. Guía de diseño

Los estudiantes deberán generar un diagrama de bloques para cada subsistema, con su funcionalidad descrita y su esquema de interconexión. Deberá presentarse adecuadamente la **ruta de datos** desde la

salida hasta la entrada, con una descripción comportamental de cada sub-bloque dentro de los subsistemas (i.e., muxes, decos, registros, etc.) similar a la Fig. 3. **No será necesario llevar la descripción a álgebra booleana.**

Cada subsistema deberá estar adecuadamente registrado a la entrada y salida. El flujo de datos debe ser indicado de manera explícita. No es necesario mostrar en detalle los bloques en que se implementen las máquinas de estados finitos (FSM) que se diseñen, pero sí sus diagramas de estado y las señales de control que manejan cada bloque en la ruta de datos.

Se utilizarán SystemVerilog y el suite de herramientas abierto usado para el primer tutorial del curso para desarrollar el sistema completo (ver [2]). Se seguirá el formato visto en clase para el estilo de codificación, cuyo apego por parte de los estudiantes formará parte de la evaluación final. Revisen especialmente la codificación de las FSM según el formato expuesto en clase.

6.1. Sobre la sincronización

Las señales externas deben sincronizarse con el reloj interno de la TangNano. Para ello, es necesario además de registrar dichas señales, eliminar potenciales rebotes.

Cada subsistema deberá estar adecuadamente registrado a la entrada y salida. El flujo de datos debe ser indicado de manera explícita. No es necesario mostrar en detalle los bloques en que se implementen las máquinas de estados finitos (FSM) que se diseñen, pero sí sus diagramas de estado y las señales de control que manejan cada bloque en la ruta de datos.

Se utilizará como señal del reloj, el pin ya provisto de 27 MHz en la TangNano. Solo podrá usarse un reloj en todo el sistema. Para los sistemas más lentos, será necesario generar las bases de tiempo por medio de divisores de frecuencia (relojes) que garanticen el adecuado refrescamiento tanto de la lectura del teclado como del despliegue de los datos.

6.2. Sobre el proceso de diseño y la verificación

Para cada subsistema, será obligatorio presentar simulaciones tanto a nivel de RTL (pre-síntesis) como con información de temporizado (post-síntesis y post-colocación-y-ruteo, las que se deberán presentar al asistente durante el horario de consulta del curso. Para ello, el grupo deberá generar las pruebas de banco o *testbenches* necesarios.

Al finalizar la primera semana de trabajo, el grupo deberá implementar al menos la primera fila del circuito de la Fig. 2 y verificar su funcionamiento a nivel de simulación ante el tutor o el profesor, antes de proceder.

Al terminar la segunda semana, los estudiantes deberán implementar el divisor completo, primero de manera combinacional y luego con el pipeline introducido, y mostrar los resultados de simulación ya sea al tutor o al profesor, antes de proceder.

6.3. Sobre la bitácora

Los estudiantes llevarán una bitácora de avance individual de los proyectos cortos. El estudiante anotará de manera manual todas mediciones y cálculos realizados durante cada experimento, desarrollo, diseño o simulación a su cargo durante el proyecto. Dicha bitácora puede ser un simple cuaderno, cuyas hojas deberán numerarse y fecharse, y que serán revisados y firmados por el asistente del curso al menos una vez por semana. El día de entrega del proyecto según el cronograma, el profesor revisará la bitácora y la podrá usar como base para evaluar el trabajo individual de cada alumno. Se deberá entregar la bitácora para revisión por parte el profesor, la cuál se devolverá calificada antes del inicio del siguiente proyecto.

6.4. Sobre el informe y la entrega

Los estudiantes deberán presentar un informe en formato *Markdown* donde se las siguientes características del diseño final (el archivo se cargará como archivo `README.md` del repositorio):

1. Una descripción general del funcionamiento del circuito completo y de cada subsistema.
2. Diagramas de bloques de cada subsistema y su funcionamiento fundamental, según descritos en la sección 6.
3. Diagramas de estado de todas las FSM diseñadas (si existen), según descritos en la sección 6.
4. Ejemplo y análisis de una simulación funcional del sistema completo, desde el estímulo de entrada hasta el manejo de los 7 segmentos.
5. Análisis de consumo de recursos en la FPGA (LUTs, FFs, etc.) y del consumo de potencia que reporta la herramienta Vivado.
6. Reporte de velocidades máximas de reloj posibles en el diseño (mínima frecuencia de reloj para este diseño: 27 MHz).
7. Análisis de principales problemas hallados durante el trabajo y de las soluciones aplicadas.

La entrega final del reporte (último día para actualizar el repositorio) y la presentación de la solución se realizarán en la fecha establecida en el cronograma del curso, a no ser que el profesor abra un plazo extra que deberá ser publicado por el TEC Digital, por el canal del curso.

El informe final de este proyecto deberá subirse en formato PDF por el sistema de evaluaciones del TEC Digital por las cuentas de ambos estudiantes (es decir, cada miembro del grupo debe subir individualmente una copia de este informe).

7. Evaluación (100 %)

La evaluación de la tarea se llevará a cabo según la siguiente rúbrica:

Funcionamiento del circuito (grupal)	60
Bitácora (individual)	20
Informe (grupal)	20
Total	100

Tabla 1: Distribución de la calificación.

Para la calificación proporcional tanto del funcionamiento en simulación como en funcionamiento del circuito armado, se aplicarán los siguiente criterios de ponderación:

El circuito funciona totalmente y las simulaciones son funcionales	100
El circuito funciona parcialmente, pero las simulaciones son totalmente funcionales	80
El circuito no funciona, pero las simulaciones son total o parcialmente funcionales	60
El circuito no funciona, y las simulaciones tampoco son funcionales del todo	25

Tabla 2: Ponderación de la calificación.

8. Puntaje extra (35 %)

Se otorgarán 35 puntos más sobre la nota para aquellos grupos que lleven el sistema hasta la capacidad de dividir al menos 127_D entre 31_D .

Para más información, revise los tutoriales y videos en [2, 1].

Referencias

- [1] David Medina. *Video tutorial para principiantes. Flujo abierto para TangNano 9k*. Jul. de 2024. URL: <https://www.youtube.com/watch?v=AKO-SaOM7BA>.
- [2] David Medina. *Wiki tutorial sobre el uso de la TangNano 9k y el flujo abierto de herramientas*. Mayo de 2024. URL: https://github.com/DJosueMM/open_source_fpga_environment/wiki.
- [3] David Money Harris y Sarah L. Harris. *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann Publishers Inc., 2022. ISBN: 978-0-12-820064-3.